



Módulo 3A — Excepciones y Fail Fast

Conceptos, fundamentos y diseño de la jerarquía de excepciones

CommerX · Arquitectura de Software Profesional · .NET 9 · Clean Architecture · DDD

Objetivo del módulo: Comprender qué son las excepciones personalizadas en DDD, por qué el principio *Fail Fast* protege la integridad del dominio, y cómo diseñar una jerarquía limpia que reemplaza los `ArgumentException` genéricos utilizados hasta el Módulo 2C en los 7 Value Objects.

1 · ¿Qué es una excepción personalizada en DDD?

Una **excepción** es un mecanismo del lenguaje que interrumpe el flujo normal de ejecución cuando ocurre una situación que el código no puede ni debe continuar procesando. No es un error de programación: es una señal intencional de que una *regla fue violada*.

En el contexto de **Domain-Driven Design**, una **excepción personalizada de dominio** es una clase que hereda de `Exception` y representa, con nombre semánticamente significativo, la violación de una regla del negocio. Es parte del lenguaje ubicuo del dominio.

Excepción genérica	<code>ArgumentException</code> , <code>Exception</code> , <code>InvalidOperationException</code> . Describe <i>qué pasó</i> a nivel técnico, no qué regla de negocio fue violada.
Excepción personalizada de dominio	<code>InvalidEmailException</code> , <code>InvalidAgeException</code> . Describe exactamente <i>qué regla falló</i> , con el valor que lo causó, y puede ser capturada con precisión por la capa Application.
Ubicación en la arquitectura	Pertenece a la capa Domain . Se crea en <code>Domain/Common/Exceptions/</code> (base) y <code>Domain/Customers/Exceptions/</code> (específicas del módulo Customer).
¿Cuándo se lanza?	Dentro del método <code>Create()</code> del Value Object, en el momento exacto en que el valor viola la regla de negocio. Nunca después .

2 · Analogía del mundo real

El detector de billetes falsos en un banco

Cuando un cajero recibe un billete sospechoso, no continúa con la transacción: detiene el proceso inmediatamente y emite una alerta específica.

Dos formas posibles de respuesta:

MALA: "Error en la transacción."

– Nadie sabe qué falló ni en qué billete.

BUENA: "El billete de \$100 con serie XZ-4421 no supera la prueba UV."

– El problema es claro, identificable y accionable.

En DDD: el billete es el "valor", el cajero es el Value Object, y la alerta específica es la excepción personalizada.

El principio Fail Fast establece que la alerta se emite en el momento del primer signo de problema – no al final.

3 · Características esenciales

Nombre semántico	El nombre de la clase expresa la regla violada: <code>InvalidEmailException</code> , <code>InvalidAgeException</code> . Al leer el nombre se comprende el problema sin abrir el mensaje.
Herencia de dominio	Todas heredan de una clase base abstracta <code>DomainException</code> que a su vez hereda de <code>Exception</code> . Esto permite capturar cualquier error de dominio con un solo <code>catch</code> .
Mensaje con el valor inválido	El constructor recibe el valor que causó la excepción e incluye ese valor en el mensaje. Ej: <code>"El valor 'abc' no es un email válido."</code>
Inmutable	Son <code>sealed</code> : no se pueden extender. El mensaje se establece en construcción y no cambia.
Sin lógica adicional	Una excepción solo transporta información sobre el error. No contiene lógica de recuperación ni acceso a datos.
Pertenecen al dominio	Se ubican en la capa Domain , no en Application ni Infrastructure. El dominio las lanza; Application las captura y las transforma.

4 · ¿Por qué se usa?

Usar `ArgumentException` en el dominio es un antipatrón porque: no forma parte del lenguaje ubicuo, no se puede capturar con precisión y obliga a parsear el mensaje de texto para identificar el error.

Identificación precisa	La capa Application puede hacer <code>catch</code> <code>(InvalidEmailException ex)</code> y saber exactamente qué pasó sin leer el mensaje.
Lenguaje ubicuo	Los nombres de las excepciones reflejan el vocabulario del negocio. Un desarrollador nuevo entiende el dominio solo leyendo los tipos.
Respuestas HTTP precisas	En el Módulo 9, el endpoint podrá devolver <code>400 Bad Request</code> con el campo exacto que falló, porque cada excepción identifica exactamente qué VO la lanzó.
Principio Fail Fast	El sistema falla inmediatamente al detectar datos inválidos, antes de persistir, procesar o propagar el error a otras partes del sistema.
Trazabilidad	En los logs se ve <code>InvalidDocumentException</code> , no un genérico <code>ArgumentException</code> difícil de diferenciar de otros errores.

El principio Fail Fast

Fail Fast establece que un sistema debe detectar y reportar errores lo más temprano posible en el flujo de ejecución. Aplicado al dominio: si el valor de un campo no es válido, la excepción se lanza *en el momento de crear el Value Object*, nunca después.

Flujo con Fail Fast:

1. Application recibe datos crudos (`string email = "usuario@mal"`)
2. Llama: `EmailAddress.Create("usuario@mal")`
3. El VO detecta el formato inválido → lanza `InvalidEmailException`
4. Application captura la excepción → devuelve respuesta 400
5. No se creó ninguna entidad
6. No se intentó persistir nada
7. El error se reportó en el paso más temprano posible

5 · ¿Cómo encaja en la arquitectura?

Clean Architecture — flujo de excepciones por capas

```

+-----+
| Value Object Create() | Lanza: InvalidEmailException
| Entidad Create()      | Lanza: InvalidAgeException
+-----+
|
v
+-----+
| Caso de Uso           | Captura: DomainException (base)
| CreateCustomerUseCase | Traduce: a respuesta de aplicación
+-----+
|
v
+-----+
| Controller / Endpoint | Mapea: excepción → HTTP 400
+-----+

```

Domain	Lanza excepciones personalizadas desde los Value Objects. No sabe nada de HTTP ni de la interfaz de usuario.
Application	Captura excepciones de dominio y las traduce al formato que necesita la capa exterior (resultado, mensaje, código). Nunca revalida lo mismo que ya validó el dominio.
Infrastructure	No captura excepciones de dominio. Si la entidad no se pudo crear, nunca llega a persistir nada.
Interface Adapter	Mapea <code>DomainException</code> → <code>400 Bad Request</code> . Puede capturar tipos específicos para mensajes más detallados.

Relación con SOLID

SRP	Cada excepción tiene una única responsabilidad: representar la violación de una regla específica. No mezcla responsabilidades.
OCP	Al agregar un nuevo VO (por ejemplo <code>Website</code>), se crea una nueva excepción <code>InvalidWebsiteException</code> sin modificar las existentes.
LSP	Todas las excepciones específicas son sustituibles por <code>DomainException</code> gracias a la herencia. Un <code>catch (DomainException)</code> captura cualquiera de ellas.
DIP	Application depende de la abstracción <code>DomainException</code> , no de cada excepción específica. Puede capturarlas todas con un solo tipo.

6 · Marco teórico complementario

Jerarquía de excepciones en .NET 9

```

System.Object
├── System.Exception (clase base de todas las excepciones)
│   ├── System.SystemException (errores del runtime)
│   │   ├── ArgumentException
│   │   ├── NullReferenceException
│   │   └── ...
│   └── System.ApplicationException (excepciones de aplicación – poco usado hoy)

```

Nuestra jerarquía personalizada:

```

System.Exception
├── DomainException (abstracta – Domain/Common/Exceptions/)
│   ├── InvalidFullNameException
│   ├── InvalidLastNameException
│   ├── InvalidDocumentException
│   ├── InvalidEmailException
│   ├── InvalidPhoneException
│   ├── InvalidAddressException
│   └── InvalidAgeException

```

¿Por qué NO usar ApplicationException como base?

Microsoft desaconseja usar `ApplicationException` como base para excepciones personalizadas desde .NET 2.0. La guía oficial indica heredar directamente de `Exception`. En nuestro diseño, `DomainException` hereda de `Exception` y sirve de base para todas las excepciones del dominio.

Estructura de carpetas resultante

```
src/
+--- Domain/
|   +--- Common/
|   |   +--- Exceptions/
|   |   |   +--- DomainException.cs          <-- NUEVA (base abstracta)
|   |   +--- Entities/
|   |   +--- ValueObjects/
|   +--- Customers/
|       +--- Entities/
|       |   +--- Customer.cs
|       +--- Exceptions/                      <-- NUEVA carpeta
|       |   +--- InvalidFullNameException.cs
|       |   +--- InvalidLastNameException.cs
|       |   +--- InvalidDocumentException.cs
|       |   +--- InvalidEmailException.cs
|       |   +--- InvalidPhoneException.cs
|       |   +--- InvalidAddressException.cs
|       +--- InvalidAgeException.cs
|       +--- ValueObjects/                    <-- MODIFICADOS (7 archivos)
|           +--- FullName.cs
|           +--- LastName.cs
|           +--- Document.cs
|           +--- EmailAddress.cs
|           +--- Phone.cs
|           +--- Address.cs
|           +--- BirthDate.cs
+--- Application/
+--- Infrastructure/
+--- InterfaceAdapter/
```

Convención de mensajes

Patrón del mensaje	"El valor '{value}' no cumple la regla: [descripción de la regla]."
Incluir el valor	Siempre incluir el valor que causó el error para facilitar el diagnóstico en logs y mensajes de respuesta.
Idioma del mensaje	Mensajes en español para el equipo actual. En sistemas internacionales se usa un código de error y localización separada.
Un constructor por excepción	Cada excepción recibe solo el valor inválido. El mensaje se construye internamente. No exponer sobrecargas innecesarias.

7 · Errores comunes

Usar ArgumentException en el dominio	Lanzar <code>ArgumentException</code> desde un VO es técnicamente válido pero semánticamente incorrecto. No forma parte del lenguaje ubicuo y no se puede capturar por tipo en la capa Application sin ambigüedad.
Crear la excepción en Application	Definir <code>InvalidEmailException</code> en la capa Application viola la arquitectura. Las reglas de validación pertenecen al dominio. Application solo captura y traduce.
Validar de nuevo en Application	Si Application vuelve a verificar el formato del email antes de llamar al VO, se duplica la lógica de negocio. Eso viola el principio Fail Fast y el SRP.
No incluir el valor en el mensaje	Un mensaje genérico como <code>"Email inválido."</code> sin el valor dificulta el diagnóstico. Los logs no indicarán qué valor causó el error.
Hacer la excepción no sellada	Si <code>InvalidEmailException</code> no es <code>sealed</code> , otra clase podría heredarlo y alterar su comportamiento. Las excepciones específicas del dominio deben ser finales.
Usar excepciones para flujo normal	Las excepciones son para condiciones excepcionales y violaciones de reglas, no para controlar el flujo normal del programa (if/else). No lanzar excepciones cuando el resultado es un caso válido esperado.
Capturar Exception a nivel de dominio	Nunca hacer <code>try/catch</code> dentro de un VO o entidad. El dominio lanza; quien traduce es Application.

8 · Qué NO hacer — código incorrecto explícito

Antipatrón 1: ArgumentException genérico en el dominio

El código siguiente usa excepciones genéricas que no pertenecen al lenguaje del dominio.

```
1 // ✗ MAL: usando ArgumentException genérico
2 public static EmailAddress Create(string value)
3 {
4     if (string.IsNullOrEmpty(value))
5         throw new ArgumentException("El email no puede estar vacío."); // ✗ genérico
6
7     if (!Regex.IsMatch(value, @"^[^@\s]+@[^@\s]+\.[^@\s]+$"))
8         throw new ArgumentException("Formato de email inválido."); // ✗ genérico
9
10    return new EmailAddress(value.Trim());
11 }
```

Antipatrón 2: validación duplicada en Application

Application vuelve a validar lo que ya hará el VO — viola SRP y Fail Fast.

```
1 // ✗ MAL: Application re-valida antes de llamar al dominio
2 public void Execute(CreateCustomerRequest request)
3 {
4     // ✗ validación duplicada — ya la hará EmailAddress.Create()
5     if (!request.Email.Contains("@"))
6         throw new Exception("Email inválido"); // ✗ duplicado y genérico
7
8     var email = EmailAddress.Create(request.Email); // el VO ya valida
9 }
```

Antipatrón 3: excepción sin el valor que causó el error

El mensaje no incluye el valor inválido — dificulta el diagnóstico.

```
1 // ✗ MAL: mensaje sin valor — inútil para diagnóstico
2 public sealed class InvalidEmailException : DomainException
3 {
4     public InvalidEmailException()
5         : base("El email no es válido.") { } // ✗ ¿qué email? ¿cuál es el valor?
6 }
```

Antipatrón 4: excepción definida fuera del dominio

```
1 // ✗ MAL: excepción en Application — viola Clean Architecture
2 // Archivo: src/Application/Exceptions/InvalidEmailException.cs
3 namespace CommerX.Application.Exceptions; // ✗ pertenece al dominio
```



```
4
5 public sealed class InvalidEmailException : Exception
6 {
7     public InvalidEmailException(string value)
8         : base($"Email inválido: {value}") { }
9 }
```

Continúa en → **Módulo 3B: Implementación completa de las 7 excepciones e integración en los Value Objects**

Módulos anteriores	Módulo 0 al 2C — completados
Módulo actual	3A — Conceptos (este documento) 3B — Implementación
Próximo módulo	Módulo 4 — Integración Entity + Value Objects